
FINAL PROJECT DOCUMENTATION

LangChain and Its Use in the NADRA RAG Assistant

Concepts, architecture, implementation, technology stack,
and project structure

NADRA RAG Project

LangChain and Its Use in the NADRA RAG Assistant

Document Purpose

This document has two major parts:

1. **What LangChain is** — its purpose, main concepts, benefits, and role in Retrieval-Augmented Generation (RAG).
 2. **How LangChain is used in this project** — an explanation of the NADRA FAQ assistant, its technology stack, structure, architecture, and the exact LangChain components used in the implementation.
-

Part I — What Is LangChain?

1. Introduction

LangChain is an open-source framework for building applications powered by Large Language Models (LLMs). An LLM can generate and understand language, but a complete application usually needs more than a model call. It may also need to load documents, divide them into manageable sections, retrieve relevant information, construct prompts, maintain metadata, and connect the model to other components.

LangChain provides standard interfaces and reusable building blocks for these tasks. It acts as an orchestration layer between an application and components such as:

- Language models
- Embedding models
- Documents and their metadata
- Text splitters
- Retrievers and vector stores
- Prompt templates
- Tools, APIs, and external data sources

LangChain is not itself an LLM or a database. It helps developers connect these technologies into a working AI application.

2. Why LangChain Is Useful

Calling an LLM directly is sufficient for a simple prompt-and-response program. A real application, however, often needs a repeatable workflow. LangChain helps by providing:

- **Standardization:** Models and data sources can be accessed through consistent interfaces.
- **Modularity:** Individual components can be replaced without redesigning the entire application.
- **Prompt management:** Dynamic prompts can be created from reusable templates.
- **Document processing:** Text can be represented, split, and passed between pipeline stages consistently.
- **Composition:** Components can be connected into chains that express the application flow clearly.
- **Integration:** The same application can combine local models, hosted models, databases, and custom Python logic.

3. Core LangChain Concepts

3.1 Documents

A LangChain `Document` contains two main elements:

- `page_content`: the actual text
- `metadata`: descriptive information such as filename, page number, category, or source

Metadata is especially important in RAG applications because it allows the system to show where an answer came from.

3.2 Text Splitters

Large documents cannot always be sent to an LLM or embedded as one unit. Text splitters divide them into smaller chunks while trying to preserve meaningful boundaries.

The `RecursiveCharacterTextSplitter`, for example, tries separators in order—such as paragraphs, lines, sentences, and spaces—until each chunk fits the configured size. A small overlap between chunks helps preserve information that crosses a chunk boundary.

3.3 Embeddings

An embedding model converts text into a numeric vector. Texts with similar meanings are expected to have vectors that are close to one another.

In a RAG system:

1. Document chunks are embedded during ingestion.
2. The user's question is embedded at query time.
3. The query vector is compared with stored document vectors.
4. The most relevant chunks are returned as context.

LangChain defines an `Embeddings` interface with operations such as `embed_documents()` and `embed_query()`. A project can implement this interface while using any suitable embedding engine underneath it.

3.4 Retrievers and Vector Stores

A vector store saves document embeddings and supports similarity search. A retriever is the application-facing component that decides which documents should be returned for a question.

Retrieval does not have to use vector similarity alone. A system can combine:

- Semantic vector search
- Keyword search such as BM25
- Metadata filtering
- Reranking
- Custom ranking or fusion logic

LangChain permits custom retrieval logic, so developers are not restricted to a specific database or algorithm.

3.5 Prompt Templates

A prompt template defines how instructions, context, and user input are presented to an LLM. Templates

make prompts consistent and allow values to be inserted at runtime.

A RAG prompt commonly contains:

- The model's role
- Rules for using retrieved information
- The retrieved context
- The user's question
- Instructions for citations or refusal behavior

3.6 Chat Models and Messages

LangChain provides chat-model interfaces for services such as Groq and for message types such as system, human, and AI messages. This gives the application a structured way to send instructions and multimodal content to a model.

3.7 Chains

A chain connects components into an executable sequence. For example:

text

```
Prompt template → Chat model → Model response
```

LangChain Expression Language supports this composition with the pipe operator:

python

```
chain = prompt | model
```

The resulting chain can then be invoked with runtime values.

4. LangChain and Retrieval-Augmented Generation

Retrieval-Augmented Generation allows an LLM to answer using a selected knowledge base instead of depending only on information learned during model training.

A typical RAG workflow has two phases.

4.1 Indexing Phase

text

```
Source documents
  ↓
Text extraction and cleaning
  ↓
LangChain Documents
  ↓
Chunking
  ↓
Embeddings
```

↓
Persistent vector index

4.2 Question-Answering Phase

text

```
User question
  ↓
Retrieve relevant chunks
  ↓
Insert chunks into a grounded prompt
  ↓
LLM generates an answer
  ↓
Return answer with source metadata
```

RAG is valuable when answers must be based on a controlled or frequently updated document collection. It can improve traceability and reduce hallucination, although its reliability still depends on document quality, retrieval quality, and prompt design.

5. What LangChain Does Not Guarantee

Using LangChain does not automatically make an AI application accurate. Developers must still:

- Use trustworthy source documents
- Clean and chunk the data properly
- Select a suitable embedding model
- Evaluate retrieval quality
- Prevent unsupported answers through prompt rules
- Handle missing or incomplete information
- Protect sensitive information

LangChain supplies the components and orchestration pattern; the project remains responsible for system design and evaluation.

Part II — How LangChain Is Used in the NADRA RAG Project

6. Project Overview

This project is a multilingual RAG-based FAQ assistant for Pakistan's National Database and Registration Authority (NADRA). It helps users find document-grounded information about:

- CNIC
- NICOP
- Pakistan Origin Card (POC)
- Child Registration Certificate (CRC)
- Family Registration Certificate (FRC)
- Related procedures such as renewal, payment, document upload, appointment scheduling, application tracking, and identity-card cancellation

Users can ask questions in English, Urdu script, Roman Urdu, or mixed language. The assistant retrieves relevant content from official NADRA PDFs and asks a hosted Llama model to answer using only that content. The interface also displays the source filename and page number.

The system is intended to provide informational guidance. It does not connect to NADRA's internal systems, submit applications, access citizen records, or track a live application directly.

7. Main Objectives

The project is designed to:

- Make official NADRA guidance easier to search
- Provide concise answers instead of requiring users to read many PDFs
- Support multilingual and code-switched questions
- Ground answers in the supplied official document collection
- Refuse questions for which verified context is unavailable
- Preserve source and page metadata for transparency
- Run embeddings and retrieval locally while using Groq only for generation and optional form transcription

8. Technology Stack

Layer	Technology	Use in the project
-------	------------	--------------------

Programming language	Python	Implements ingestion, retrieval, generation, evaluation, CLI, and UI
LLM framework	LangChain Core and LangChain Text Splitters	Documents, embedding interface, messages, prompts, chain composition, and chunking
LLM integration	<code>langchain-groq</code>	Connects LangChain to Groq-hosted language and vision models
Answer-generation model	Llama 3.3 70B Versatile through Groq	Produces grounded answers from retrieved context
Vision model	Llama 4 Scout through Groq	Optionally transcribes image-only NICOP and POC form PDFs
Embedding model	<code>intfloat/multilingual-e5-small</code>	Creates 384-dimensional multilingual document and query vectors
Embedding runtime	FastEmbed and ONNX Runtime	Runs the embedding model locally on CPU
Semantic retrieval	NumPy cosine similarity	Compares a question vector with stored document vectors
Keyword retrieval	BM25 through <code>rank-bm25</code>	Finds exact terms, acronyms, form names, and other keyword-heavy matches
Ranking fusion	Reciprocal Rank Fusion (custom Python)	Combines vector-search and BM25 rankings
PDF processing	PyMuPDF (<code>fitz</code>)	Extracts PDF text and renders scanned pages as images
Persistent knowledge base	JSON and NumPy <code>.npy</code> files	Stores chunks, metadata, embeddings, and the index manifest locally
Web interface	Streamlit	Provides the browser-based chat experience
Command-line interface	Python terminal application	Provides a lightweight interactive test interface
Configuration	<code>python-dotenv</code>	Loads <code>GROQ_API_KEY</code> from <code>.env</code>
Evaluation	Python, JSON, and Groq LLM-as-judge	Measures retrieval hits, refusal behavior, and answer correctness

The current implementation does **not** use ChromaDB. Some earlier planning and analysis files refer to ChromaDB and an English MiniLM embedding model, but the current source code uses a custom local NumPy vector store and the multilingual E5 model.

9. Project Structure

text

```

NADRA-RAG-main/
|
├── app.py
|   Streamlit chat interface. Displays conversations and source references.

```



```

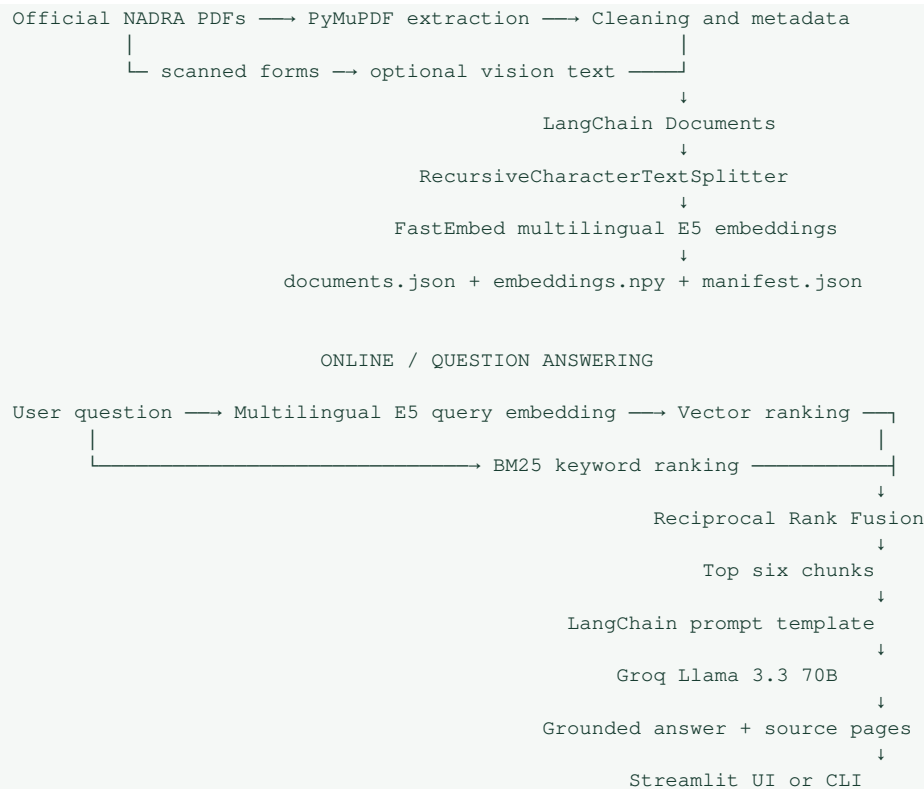
├── requirements.txt
│   Python runtime dependencies.
├── .env
│   Local API configuration, especially GROQ_API_KEY. It should not be committed.
├── pdfs_data/
│   Official NADRA PDF knowledge sources.
│   ├── transcribed/
│   │   Optional text sidecars produced for scanned form PDFs.
├── data/
│   Locally generated models and knowledge-base artifacts.
│   ├── fastembed_cache/
│   │   Cached ONNX embedding model files.
│   ├── vector_store/
│   │   ├── documents.json
│   │   │   Chunk text and source metadata.
│   │   ├── embeddings.npy
│   │   │   Stored document vectors.
│   │   └── manifest.json
│   │       Embedding model and vector-dimension information.
├── src/
│   ├── kb.py
│   │   Shared paths, embedding adapter, local vector store, BM25 search,
│   │   and hybrid retrieval.
│   ├── ingest.py
│   │   Extracts and cleans PDFs, creates LangChain Documents, splits them
│   │   into chunks, embeds them, and builds the persistent knowledge base.
│   ├── rag_pipeline.py
│   │   Retrieves context, builds the prompt, calls the Groq chat model,
│   │   and returns an answer with sources.
│   ├── chat_cli.py
│   │   Command-line chat client for the RAG pipeline.
│   └── transcribe_forms.py
│       Optional one-time vision transcription for image-only PDFs.
├── evaluation/
│   ├── test_questions.json
│   │   Questions, expected sources, reference answers, and refusal labels.
│   └── evaluate.py
│       Evaluation script for retrieval, refusal, and generation quality.
├── TEST_QUESTIONS.md
│   Manual test cases and expected behavior.
├── PLAN.md
│   Initial project plan; parts of it describe the earlier architecture.
├── ANALYSIS.md
├── IMPROVEMENTS.md
└── CODE_CHANGES.md
    Historical analysis and implementation notes.

```

Generated directories can change after ingestion. In the inspected repository snapshot, the local index contains 311 chunks from a collection of 25 PDFs, using 384-dimensional multilingual E5 embeddings.

10. System Architecture

text		
OFFLINE / INGESTION		



11. How LangChain Is Used During Ingestion

11.1 Representing Extracted Pages as Documents

In `src/ingest.py`, every usable PDF page is converted into a LangChain `Document`:

python

```
Document (
  page_content=content,
  metadata={
    "source": pdf_path.name,
    "service": service,
    "page": page_num,
  },
)
```

This gives every text unit a consistent representation. The metadata later enables service identification and human-readable citations.

11.2 Splitting Documents into Chunks

The project uses LangChain's `RecursiveCharacterTextSplitter`:

python

```
splitter = RecursiveCharacterTextSplitter(
  chunk_size=1000,
  chunk_overlap=150,
  separators=["\n\n", "\n", ". ", " ", ""],
)
```

The settings mean:

- A chunk is targeted at no more than 1,000 characters.
- Adjacent chunks overlap by 150 characters.
- Paragraph and line boundaries are preferred before sentences or spaces.
- Very short chunks under 60 characters are removed.

The splitter preserves the original `Document` metadata in the resulting chunks.

11.3 Using the LangChain Embeddings Interface

`src/kb.py` defines `FastEmbedEmbeddings`, which subclasses LangChain's `Embeddings` interface:

python

```
class FastEmbedEmbeddings(Embeddings):
    def embed_documents(self, texts):
        ...

    def embed_query(self, text):
        ...
```

Under the interface, `FastEmbed` executes the multilingual E5 model locally through ONNX Runtime.

Documents receive the E5 `passage:` prefix, while user questions receive the `query:` prefix, following the model's retrieval format.

This is an example of LangChain's modular design: the project keeps a standard LangChain embedding interface while using a custom local storage and retrieval implementation.

12. How Retrieval Works

The retriever in `src/kb.py` is custom and combines two methods.

12.1 Semantic Vector Search

The local vector store:

1. Embeds the question.
2. Normalizes the question vector.
3. Computes its dot product with normalized document vectors.
4. Selects the highest-scoring chunks.

With normalized vectors, the dot product corresponds to cosine similarity.

Semantic search is helpful when the question and document express the same idea with different words. Multilingual E5 also lets Urdu or Roman-Urdu questions retrieve relevant English passages.

BM25 searches the same chunks using keywords. The project applies:

- Lowercasing
- Stopword removal
- Simple suffix stripping

This improves matching for variations such as “renew” and “renewal.” BM25 is particularly useful for exact service names, acronyms, and form-related terms that semantic search may underweight.

12.3 Reciprocal Rank Fusion

The two result lists are combined through Reciprocal Rank Fusion. Each document receives a contribution based on its position in each ranking:

text

```
score += 1 / (60 + rank)
```

The combined results are sorted, and the top six chunks are sent to the language model. This hybrid design improves retrieval without requiring vector and BM25 scores to use the same numerical scale.

13. How LangChain Is Used for Answer Generation

13.1 Building the Prompt

`src/rag_pipeline.py` uses `ChatPromptTemplate` to construct a system message and a human message:

python

```
prompt = ChatPromptTemplate.from_messages(
    [{"system", SYSTEM_PROMPT}, {"human", "{question}"}]
)
```

The system prompt contains the retrieved context and important safety rules:

- Answer only from the supplied context.
- Do not invent information.
- Refuse when no verified information is available.
- Provide verified partial information when only part of a question is covered.
- Match the user's language and writing style.
- Preserve exact fees, timelines, and document lists.
- Include document names and page numbers.

13.2 Connecting the Prompt to Groq

The project initializes the model through LangChain's Groq integration:

python

```
ChatGroq(
```

```
model="llama-3.3-70b-versatile",
temperature=0,
)
```

A temperature of zero is used to favor consistent, less-random answers.

The prompt and model are composed into a chain:

python

```
chain = prompt | llm
response = chain.invoke({
    "context": formatted_context,
    "question": question,
})
```

This is the project's clearest use of LangChain as an orchestration framework.

13.3 Returning Sources

The pipeline returns:

python

```
{
    "answer": response.content,
    "sources": [
        {"source": "...", "page": 1},
        ...
    ],
}
```

The Streamlit interface displays these sources inside an expandable section. Source data comes from the metadata initially attached to each LangChain `Document`.

14. LangChain in Scanned-Form Transcription

Some PDFs may be image-only and contain no extractable text. `src/transcribe_forms.py` can:

1. Render each scanned page as a PNG using PyMuPDF.
2. Encode the image in base64.
3. Create a LangChain `HumanMessage` containing text and image content.
4. Send the message through `ChatGroq` to a vision-capable Llama model.
5. Save the transcription as a text sidecar for later ingestion.

The sidecar approach prevents repeated vision-model calls whenever the index is rebuilt.

15. User Interfaces

15.1 Streamlit Application

`app.py` provides:

- A browser-based chat interface
- Example NADRA questions
- Conversation history
- English, Urdu, and Roman-Urdu input
- Expandable source citations
- A privacy reminder not to share CNIC numbers, biometric data, passwords, or other sensitive information
- Cached knowledge-base warm-up at application startup

15.2 Command-Line Interface

`src/chat_cli.py` provides a terminal loop that calls the same `answer_question()` function. It is useful for development and manual testing without starting the web interface.

Both interfaces remain thin layers over the same RAG pipeline, avoiding duplicated retrieval or generation logic.

16. Evaluation

`evaluation/evaluate.py` is designed to measure three aspects:

1. **Retrieval hit rate:** whether an expected PDF appears among the retrieved chunks.
2. **Refusal accuracy:** whether the system refuses out-of-scope questions.
3. **Answer correctness:** whether a Groq-hosted judge model considers an answer consistent with a manually verified reference answer.

The starter evaluation file currently contains ten questions, and several reference answers are still marked `TODO`. Therefore, the evaluation framework exists, but a complete final accuracy result should not be claimed until the reference set is expanded and verified.

17. Key Design Decisions

17.1 Multilingual Retrieval Over English Passages

The PDFs contain English and Urdu, but the Urdu text is often a translation of the English and may have broken font mappings. The ingestion process keeps clean Latin-script content, while multilingual E5 maps English, Urdu, and mixed-language questions into a shared vector space. The LLM then responds in the user's language.

17.2 Hybrid Instead of Vector-Only Retrieval

Vector search is strong at meaning and paraphrases. BM25 is strong at exact words and acronyms. Combining them gives the system better coverage than either method alone.

17.3 Local Embeddings and Storage

Document embedding and retrieval run locally. This:

- Avoids per-query embedding API costs
- Keeps the document index on the local machine
- Allows the index to be rebuilt from the supplied PDFs
- Reduces dependence on external services

Groq is still required for final answer generation and for optional vision transcription.

17.4 Strict Grounding

Government-service information can be sensitive to accuracy. The model is explicitly instructed to use only retrieved context and to admit when verified information is missing. This is preferable to producing a confident but unsupported answer.

18. Current Limitations

- The assistant is informational and has no access to live NADRA systems.
- Answer quality is limited by the contents and freshness of the PDF collection.
- The corpus does not contain a complete fee schedule, so some fee questions must be declined or only partially answered.
- Fine-grained fields buried in long form sections may be difficult to retrieve.
- The evaluation set is still a starter set and lacks verified reference answers for several questions.
- Hybrid retrieval uses an English-oriented BM25 tokenizer; multilingual semantic retrieval handles Urdu questions, but Urdu keyword matching is limited.
- The application sends the question and retrieved context to Groq for answer generation, so production deployment should include an appropriate privacy and data-handling review.

19. End-to-End Example

For a question such as:

What documents are required to renew a CNIC?

the system performs the following steps:

1. The Streamlit or CLI interface sends the question to `answer_question()`.
2. Multilingual E5 converts the question into a 384-dimensional vector.
3. The local vector store finds semantically similar chunks.
4. BM25 separately ranks chunks using keyword matches.
5. Reciprocal Rank Fusion combines the rankings.
6. The top six chunks are formatted with source filenames and page numbers.

7. `ChatPromptTemplate` inserts the context and question into the grounded prompt.
8. The LangChain Groq integration sends the prompt to Llama 3.3 70B.
9. The model generates an answer using only the retrieved evidence.
10. The interface displays the answer and its source references.

20. Conclusion

LangChain provides the common AI-application layer in this project. It represents source material as documents, splits content into retrieval chunks, defines the embedding interface, structures multimodal messages, builds grounded prompts, connects the application to Groq, and composes prompt and model components into an executable chain.

The wider system combines these LangChain components with custom engineering: PyMuPDF-based extraction, text cleaning, multilingual E5 embeddings through FastEmbed, a NumPy vector store, BM25 keyword search, Reciprocal Rank Fusion, Streamlit, and an evaluation framework.

The result is a focused multilingual RAG assistant that makes official NADRA documents easier to query while preserving source traceability and refusing to invent information that is not supported by the available documents.